

Full-Stack Developer Interview Question Bank

FRONTEND ARCHITECTURE • BACKEND MICROSERVICES • SYSTEM DESIGN

Target Role: Full-Stack Software Engineer

Total Questions: 80 (10 per Level per Domain)

Domain 1: Frontend Architecture & UI State Engineering

LEVEL 1: FOUNDATIONAL

1. What is the difference between the Virtual DOM and the real DOM, and how does reconciliation work in modern frameworks?
2. How do HTTP methods (GET, POST, PUT, PATCH, DELETE) differ in terms of idempotency and safety?
3. What is the JavaScript Event Loop, and how do Call Stack, Web APIs, Microtasks (Promises), and Macrotasks interact?
4. What is the difference between client-side rendering (CSR), server-side rendering (SSR), and static site generation (SSG)?
5. What are Closures in JavaScript, and how do they enable data encapsulation?
6. What is Event Bubbling and Event Capturing, and how does event delegation work?
7. How do CSS Flexbox and CSS Grid differ in layout rendering mechanics?
8. What is the difference between var, let, and const in terms of scope and hoisting?
9. What are Web Workers, and how do they enable multithreaded processing in the browser?
10. What is Semantic HTML, and why is it critical for accessibility and web performance?

LEVEL 2: INTERMEDIATE / PROJECT & RESUME BASED

11. How do you optimize initial page load performance metrics (LCP, FID/INP, CLS) on a resource-heavy Single Page Application (SPA)?
12. How do you manage complex global UI state using Context API, Redux Toolkit, or Zustand while preventing unnecessary re-renders?
13. Walk through how you implemented JWT authentication with access and refresh tokens on the client side securely against XSS and CSRF attacks.
14. How do you implement dynamic code splitting, lazy loading, and tree-shaking using Webpack/Vite bundlers?
15. Describe how you handle asynchronous state synchronization, optimistic UI updates, and caching using tools like React Query / TanStack Query.
16. How do memoization hooks (e.g., useMemo, useCallback, React.memo) optimize component render lifecycles?
17. How do you construct accessible UI components complying with WAI-ARIA standards and screen readers?
18. How do debouncing and throttling techniques optimize high-frequency events like search inputs or window scrolling?
19. How do you setup automated end-to-end (E2E) testing suites using Cypress or Playwright for critical UI workflows?
20. How do CSS-in-JS libraries (e.g., Styled Components, Emotion) compare to utility-first CSS (Tailwind) in bundle size and performance?

LEVEL 3: ADVANCED / PRODUCTION & SCENARIO BASED

21. How would you architect a Micro-Frontend architecture for a large enterprise codebase managed by multiple autonomous engineering teams?

22. How do you handle memory leaks in large single-page applications caused by detached DOM nodes, uncleaned event listeners, or subscriptions?
23. Design a real-time collaborative frontend UI (like Notion or Figma) handling multi-user cursor tracking and WebSocket reconnect logic.
24. How do you build a progressive web app (PWA) with Service Workers that operates seamlessly in offline mode using IndexedDB state sync?
25. How do you handle Server-Driven UI (SDUI) architecture where layout, state, and components are dynamically controlled by backend payloads?
26. How do React Server Components (RSC) alter the traditional client-server boundary and data fetching paradigms?
27. How do you render large dataset tables with 100,000+ rows smoothly using DOM virtualization techniques (e.g., react-window)?
28. How do you structure Cross-Origin Resource Sharing (CORS) and Content Security Policy (CSP) headers to prevent XSS injection?
29. How do you design resilient error boundary architectures to gracefully isolate visual component failures without crashing the whole application?
30. How do WebSockets, Server-Sent Events (SSE), and Long Polling compare for building real-time notification feeds?

LEVEL 4: EXPERT / HIGH-PRESSURE & STRATEGY SCENARIOS

31. During a high-traffic sale event, your client application experiences severe UI freeze frame drops for thousands of users. How do you profile and patch the runtime bottleneck live?
32. How do you design an automated Web Vitals performance budget pipeline integrated into GitHub Actions CI/CD to block performance regression PRs?
33. How do you lead a complete zero-downtime frontend framework migration (e.g., Angular to React/Next.js) for an enterprise application receiving millions of monthly visits?
34. A critical third-party NPM dependency maintained deep in your frontend tree releases a compromised malicious update. How do you respond and audit runtime assets?
35. How do you architect a global WebAssembly (WASM) module integration to run heavy computational desktop-class algorithms natively inside the browser?
36. How do you implement edge rendering architectures (e.g., Cloudflare Workers, Vercel Edge) to deliver sub-10ms SSR rendering globally?
37. How do you design cross-platform design system token pipelines synchronizing Figma assets directly into React, iOS, and Android target codebases?
38. How do you isolate and fix hydration mismatch errors occurring during high-concurrency Server-Side Rendering (SSR)?
39. How do you establish enterprise frontend observability tracking client-side runtime errors, telemetry, and network latency traces at scale?
40. How do you design an A/B testing infrastructure at the CDN layer to prevent layout shifts (CLS) and flashing unstyled content?

Domain 2: Backend Microservices, API Design & System Architecture

LEVEL 1: FOUNDATIONAL

41. What are the core differences between REST, GraphQL, and gRPC API communication protocols?
42. What is ACID compliance in relational databases versus BASE model in NoSQL databases?
43. What is connection pooling in backend microservices, and why is it necessary?

- 44. What is the difference between synchronous and asynchronous backend task execution?
- 45. What is CORS (Cross-Origin Resource Sharing), and how does the browser enforce preflight OPTIONS requests?
- 46. What is the difference between monolithic architecture and microservices architecture?
- 47. What is an API Gateway, and what core responsibilities does it handle?
- 48. What is the purpose of environment variables and configuration management in cloud deployment?
- 49. How do stateful vs. stateless server architectures handle horizontal scaling?
- 50. What is Object-Relational Mapping (ORM), and what are the trade-offs of using ORMs over raw SQL queries?

LEVEL 2: INTERMEDIATE / PROJECT & RESUME BASED

- 51. How do you implement API rate limiting and throttling (e.g., Token Bucket, Leaky Bucket algorithms) to prevent DDoS attacks?
- 52. Describe a scenario where you implemented database caching using Redis/Memcached to reduce backend API latency.
- 53. How do you structure database migration scripts (e.g., Prisma, Knex, Flyway) without causing service downtime during deployment?
- 54. Walk through how you design an asynchronous message queue system using RabbitMQ or Kafka for background job processing.
- 55. How do you prevent SQL Injection, Cross-Site Scripting (XSS), and Remote Code Execution (RCE) in full-stack backend handlers?
- 56. How do you resolve the GraphQL N+1 problem using DataLoader batching and caching patterns?
- 57. How do you implement database indexing strategies to optimize complex multi-column filter queries?
- 58. How do you set up structured logging and distributed request tracing (e.g., Winston, OpenTelemetry) across microservices?
- 59. How do you handle password hashing, salting, and storage securely using algorithms like bcrypt or Argon2?
- 60. How do you handle graceful shutdown signals (SIGTERM/SIGINT) in Node.js/Go backend web servers?

LEVEL 3: ADVANCED / PRODUCTION & SCENARIO BASED

- 61. Design a high-availability URL shortener system (like Bitly) capable of handling 10,000 writes/sec and 100,000 reads/sec with low latency.
- 62. How do you implement distributed transaction management across microservices using the Saga Pattern (Choreography vs. Orchestration)?
- 63. Explain how you debug and mitigate deadlocks, race conditions, and thread contention in high-throughput backend applications.
- 64. How do you architect database sharding and read-replica replication logic when a single database cluster exceeds write capacity?
- 65. How do you implement Circuit Breaker patterns (e.g., Resilience4j, Hystrix) to prevent cascading failures across backend services?
- 66. Design an enterprise event-driven architecture using Event Sourcing and CQRS (Command Query Responsibility Segregation).
- 67. How do you secure server-to-server microservice communications using mutual TLS (mTLS) and service meshes like Istio?
- 68. How do you design idempotency keys into payment processing APIs to prevent duplicate charges during network retries?

- 69. How do you structure database connection pool parameters (min/max connections, timeout thresholds) under extreme spikes?
- 70. How do you implement multi-tenant database isolation strategies (Row-Level Security vs. Schema-per-Tenant vs. DB-per-Tenant)?

LEVEL 4: EXPERT / HIGH-PRESSURE & STRATEGY SCENARIOS

- 71. A primary database node fails during peak traffic. The failover replica experiences a 15-minute data replication lag. How do you recover state consistency?
- 72. How do you design an enterprise CI/CD deployment strategy utilizing Blue-Green or Canary releases with automated rollback telemetry triggers?
- 73. How do you scale containerized backend services using Kubernetes auto-scaling (HPA/VPA) driven by custom application performance metrics?
- 74. A thundering herd problem occurs as 100,000 requests hit your database simultaneously after a cache eviction. How do you resolve and prevent it?
- 75. How do you execute a live zero-downtime database schema refactoring involving splitting a high-traffic table into two normalised tables?
- 76. How do you design multi-region active-active database replication architectures handling concurrent write conflict resolutions?
- 77. Your system experiences severe memory leak crashes in production microservices under high load. What is your step-by-step memory heap dump analysis protocol?
- 78. How do you architect a high-throughput notification engine delivering 10 million personalized push/email notifications per hour?
- 79. How do you establish Chaos Engineering practices (e.g., Chaos Mesh, Gremlin) to proactively validate system fault tolerance?
- 80. How do you design disaster recovery strategies to meet strict Recovery Point Objectives (RPO) and Recovery Time Objectives (RTO)?